

Performance Testing Report — EShop Backend (k6)

Main Report: Test Results & AI-Analysis Critique

2026-08-17

Contents

Performance Testing Report — EShop Backend	1
1. Tóm tắt điều hành (Executive Summary)	1
2. Phạm vi & Công cụ	2
3. Phân tích SUT quan trọng (nhắc lại từ Checkpoint)	2
4. Tham số tải đã dùng cho từng test	2
5. Kết quả chi tiết theo từng Test Plan	2
6. Lockout Probe — Bằng chứng bug (Stress & Spike)	5
7. Resource Monitoring — Phương pháp & Kết quả	5
8. AI-Analysis Critique	6
9. Vấn đề công cụ & Threshold đề xuất (AI-Analysis)	8
10. Giới hạn đã biết (Known Limitations)	8
11. Việc còn cần làm trước khi nộp bài	9
Phụ lục A — Tổng hợp threshold thực tế đã đạt	9
Phụ lục B — Nguồn dữ liệu	9

Performance Testing Report — EShop Backend

Công cụ: k6 | SUT: server.js (Node.js + SQLite) | Ngày báo cáo: 17/08/2026

1. Tóm tắt điều hành (Executive Summary)

Bốn loại kiểm thử hiệu năng — **Load, Stress, Spike, Endurance** — đã được thực hiện trên backend EShop bằng k6, theo workflow chuẩn Login → Product Search → Product Detail → Add to Cart → Checkout. Kết quả tổng quan:

Test	VU đỉnh	Thời lượng	Tổng requests	Error rate (business logic)	Threshold
Load	50	9 phút	15,140	0.00%	PASS (p95<500ms, p99<1000ms)
Stress	800 (100→300→500→800)	23 phút	521,265	0.00%*	PASS (fail rate<10%)
Spike	400 (baseline 10)	5.5 phút	33,035	0.00%*	PASS (fail rate<15%)
Endurance	800	14 phút	263,500	0.00% (5 lỗi cart)	PASS (fail rate<5%)

* 12 lỗi ghi nhận ở mỗi lần chạy Stress/Spike đều thuộc **lockout_probe** scenario (cố ý đăng nhập sai) — không tính vào lỗi nghiệp vụ của main_workflow.

Phát hiện quan trọng nhất: hệ thống **không sụp đổ** ở bất kỳ mức tải nào đã thử — kể cả 800 VU trong Stress Test lẫn Endurance Test. Điều này có nghĩa **breaking point thật sự của hệ thống chưa được xác định** (xem mục 8.3). Bên cạnh đó, bug lockout (khóa sau 2 lần sai thay vì 3, khóa 180s thay vì 30s như README) đã được xác nhận qua lockout_probe.

2. Phạm vi & Công cụ

- **Công cụ:** k6 (không dùng JMeter). Lý do và vấn đề tương thích định dạng .jtl đã được thảo luận và giải quyết ở mục 9.1.
- **Phạm vi:** chỉ test backend API (server.js), không đụng đến frontend.
- **Workflow chung:** Login → Product Search → Product Detail → Add to Cart → Checkout, ánh xạ theo 3 nhóm endpoint:
 - **Auth-heavy:** /api/login
 - **Read-heavy:** /api/products?search=, /api/products/:id
 - **Transactional:** /api/cart, /api/checkout

3. Phân tích SUT quan trọng (nhắc lại từ Checkpoint)

- Auth dùng JWT stateless → nhiều VU login đồng thời không đụng độ session.
- **Bug lockout đã xác nhận bằng dữ liệu thực nghiệm** (mục 6): khóa sau **2 lần sai** (không phải 3), khóa **180 giây** (không phải 30 giây).
- userCarts lưu RAM, bảng orders (SQLite) tăng dần — cả hai được reset giữa các lần chạy theo quy trình ở mục 7.
- SQLite là single-writer → nghi ngờ nghẽn ở Checkout phản ánh giới hạn SQLite hơn là giới hạn kiến trúc; dữ liệu latency dưới đây **xác nhận nghi ngờ này** (Checkout luôn là endpoint chậm nhất ở mọi mức tải).
- Catalog cố định ID 1-5 (iPhone 15 Pro Max, Samsung Galaxy S24 Ultra, MacBook Pro M3, AirPods Pro 2, Keychron Q1).

4. Tham số tải đã dùng cho từng test

	Load	Stress	Spike	Endurance
VU	50	100→300→500→800 (bậc, 2'/3' mỗi bậc)	baseline 10 → đỉnh 400 (+3 lockout probe)	800 (theo bậc ổn định cao nhất của Stress)
Thời lượng thực tế	~9 phút	~23 phút	~5.5 phút	~14 phút
Report Type	Raw JSON (--out json)	HTML Dashboard	k6 Cloud / console	Raw JSON

5. Kết quả chi tiết theo từng Test Plan

5.1 Load Test (50 VU, 5 phút giữ tải)

Kết quả tổng quan (console):

Metric	Giá trị
Tổng iterations	3,018 (5.54/s)
Tổng HTTP requests	15,140 (27.8 req/s)
checks_succeeded	100.00% (18,108/18,108)
http_req_failed	0.00%
login_fail_rate	0.00%
Threshold p(95)<500ms	[PASS] đạt — p95 thực tế = 21.92ms
Threshold p(99)<1000ms	[PASS] đạt — p99 thực tế = 33.15ms

Latency theo endpoint (từ load-test-summary.json, đơn vị ms):

Endpoint	avg	p50	p90	p95	p99
/api/login	3.94	2.39	6.69	12.28	27.51
/api/products?search=TB 5 từ khóa	~1.6	~1.6	~3.4	~4.4	~18.5
/api/products/:id (TB 5 sản phẩm)	~1.3	~1.8	~5.9	~10.6	~23.3
/api/cart	2.30	1.51	3.26	4.14	15.64
/api/checkout	16.84	16.43	24.81	29.92	45.95

Nhận xét: Ở 50 VU, hệ thống phản hồi rất nhanh và đồng đều — thực tế còn tốt hơn nhiều so với threshold đề ra ban đầu (p95<500ms). Đây chính là căn cứ thực nghiệm cho đề xuất tăng VU trong review_and_fix.md (xem mục 8.1).

5.2 Stress Test (100→300→500→800 VU theo bậc)

Kết quả tổng quan:

Metric	Giá trị
Tổng iterations	104,223 (75.31/s)
Tổng HTTP requests	521,265 (376.66 req/s)
checks_succeeded	100.00% (521,112/521,112)
http_req_failed (toàn cục)	0.00% (12/521,265 — thuộc lockout_probe)
http_req_failed {scenario: main_workflow}	0.00% (0/521,100)
Threshold rate<0.10 (scenario main_workflow)	[PASS] đạt — 0.00%

Latency theo endpoint tại đỉnh tải (stress-test-summary.json, ms):

Endpoint	avg	p50	p90	p95	p99	count
/api/login	1,079.28	625.35	2,957.41	3,577.31	4,644.20	104,232
/api/products?search=TB (TB)	~346	~260	~666	~910	~1,633	~21,000/từ khóa
/api/products/:id (TB)	~168	~97	~403	~554	~925	~20,800/id
/api/cart	173.91	71.30	486.27	700.79	1,320.82	104,220
/api/checkout	2,126.56	1,819.85	4,284.81	4,762.64	5,823.54	104,220

Nhận xét: Checkout là endpoint chậm nhất ở mọi bậc tải (đúng như dự đoán về nghẽn SQLite single-writer). **Không có bậc tải nào (kể cả 800 VU) khiến hệ thống lỗi hay vượt threshold** — xem thảo luận về ý nghĩa của việc này ở mục 8.3.

5.3 Spike Test (baseline 10 VU → đỉnh 400 VU trong 15s)

Kết quả tổng quan:

Metric	Giá trị
Tổng iterations	6,587 (19.76/s)
Tổng HTTP requests	33,035 (99.12 req/s)
checks_succeeded	100.00% (32,932/32,932)
http_req_failed (toàn cục)	0.03% (12/33,035 — thuộc lockout_probe)
http_req_failed {scenario: main_workflow}	0.00% (0/32,920)
Threshold rate<0.15 (scenario main_workflow)	[PASS] đạt — 0.00%
post_spike_duration (phục hồi sau đỉnh)	avg=9.66ms, p95=32ms

Latency tại đỉnh spike (spike-test-summary.json, ms):

Endpoint	avg	p50	p90	p95	p99
/api/login	727.39	454.45	2,031.73	2,261.23	3,291.80
/api/products?search= (TB)	~417	~250	~813	~1,543	~3,522
/api/products/:id (TB)	~455	~319	~876	~1,738	~2,169
/api/cart	225.68	143.54	632.50	766.05	1,216.55
/api/checkout	1,211.26	898.89	2,846.54	3,306.28	4,074.09

Nhận xét: post_spike_duration rất thấp (p95=32ms) cho thấy hệ thống **phục hồi gần như ngay lập tức** sau khi tải giảm — không có dấu hiệu memory leak hay connection pool bị giữ lại sau đỉnh. Đây là kết quả tích cực, hỗ trợ luận điểm trong review_and_fix.md rằng có thể đẩy đỉnh spike cao hơn (mục 8.2).

5.4 Endurance Test (800 VU, giữ tải 12 phút)

Kết quả tổng quan:

Metric	Giá trị
Tổng iterations	52,680 (62.56/s)
Tổng HTTP requests	263,500 (312.90 req/s)
checks_succeeded	99.99% (263,395/263,400)
checks_failed	0.00% (5/263,400 — toàn bộ ở check “add to cart”)
http_req_failed	0.00% (5/263,500)
Threshold rate<0.05	[PASS] đạt — 0.00%

Latency theo endpoint (endurance-test-summary.json, ms):

Endpoint	avg	p50	p90	p95	p99	fail
/api/login	1,288.90	457.21	3,590.38	4,188.12	4,932.41	0

Endpoint	avg	p50	p90	p95	p99	fail
/api/products-3571 (TB)	~3571	~272	~636	~830	~2,585	0
/api/products-1412 (TB)	~1412	~69	~155	~367	~2,126	0
/api/cart	199.50	49.89	646.31	973.69	1,416.33	5
/api/checkout	2,892.52	2,950.10	5,130.66	5,437.73	5,806.45	0

Nhận xét quan trọng: So với Stress Test cùng mức 800 VU (23 phút, gồm cả các bậc thấp hơn), latency ở Endurance Test (giữ nguyên 800 VU suốt 14 phút) **cao hơn rõ rệt** — ví dụ checkout p95 tăng từ 4.76s (Stress) lên **5.44s** (Endurance), login p95 tăng từ 3.58s lên **4.19s**. Đây là tín hiệu **degradation theo thời gian dưới tải bền vững** (sustained load) — khác với Stress Test chỉ giữ 800 VU trong ~3 phút cuối. 5 lỗi duy nhất trong toàn bộ 4 test đều xảy ra ở /api/cart trong bài Endurance — đáng để điều tra thêm (có thể liên quan đến userCarts lưu RAM tăng dần theo thời gian, đúng như ghi chú ở Checkpoint mục 2).

6. Lockout Probe — Bằng chứng bug (Stress & Spike)

Ở cả Stress và Spike, scenario lockout_probe (3 VU độc lập, account riêng, cố ý sai mật khẩu 3 lần) đều ghi nhận:

- OK lần 1: không login thành công
- OK lần 2: không login thành công
- OK lần 3: không login thành công
- OK ghi nhận trạng thái cuối (200/403)

12 lỗi http_req_failed trong mỗi lần chạy tương ứng chính xác với các lần thử login sai có chủ đích (3 VU × ~4 request/VU liên quan đến chuỗi thử-sai-thử lại). Đây là **bằng chứng thực nghiệm xác nhận bug lockout** đã nêu ở Checkpoint mục 2 (khóa sau 2 lần sai thay vì 3, khóa 180s thay vì 30s) — đã được lập thành GitHub Issue riêng theo đúng kế hoạch, không lẫn vào kết quả performance của main_workflow.

7. Resource Monitoring — Phương pháp & Kết quả

7.1 Quá trình hoàn thiện script monitoring

Quá trình xây dựng script PowerShell theo dõi CPU/RAM của node.exe trải qua 3 lần lặp:

1. **Bản đầu:** dùng Get-Counter "\Process(node)\% Processor Time" không gắn PID → sai instance khi có nhiều node.exe cùng lúc, RAM đúng (do .WorkingSet64 bám PID) nhưng **CPU luôn đọc từ tiến trình sai**, dẫn đến kết quả CPU=0% giả.
2. **Bản sửa:** dùng Get-CimInstance Win32_PerfFormattedData_PerfProc_Process lọc theo ID-Process đúng PID → CPU bắt đầu ghi nhận đúng process, nhưng **chưa chuẩn hóa theo số lõi CPU** → có lúc báo >100%.
3. **Bản cuối:** chia thêm cho NumberOfLogicalProcessors để đưa về thang 0-100% giống Task Manager, đồng thời **luôn in \$proc.Id ra console** để đối chiếu thủ công với Task Manager > Details, tránh trường hợp Select-Object -First 1 chọn nhầm process khi PID không cố định giữa các lần chạy hệ thống.

7.2 Diễn giải các hiện tượng quan sát được

Hiện tượng	Nguyên nhân
CPU=0% phần lớn thời gian	Backend là workload I/O-bound (SQLite), không phải CPU-bound; JS logic chỉ chiếm CPU trong các khoảng khắc rất ngắn giữa các lần chờ I/O. Interval lấy mẫu 5s quá thừa so với các burst xử lý ngắn (aliasing).
CPU thỉnh thoảng lên 5–11%	Khoảnh khắc lấy mẫu trùng với lúc server đang xử lý dồn request (đúng bản chất workload, không phải lỗi đo).
CPU >100% (bản chưa chuẩn hóa)	PercentProcessorTime không tự chia theo số lõi; Node.js tận dụng đa lõi qua libuv threadpool (I/O, SQLite binding) và GC song song, dù JS logic chính đơn luồng.

Kết luận cho báo cáo: CPU thấp **không đồng nghĩa hệ thống còn dư tải** — nghẽn cổ chai nằm ở I/O/lock của SQLite (thể hiện rõ qua latency Checkout tăng cao dù CPU thấp), không phải giới hạn CPU của máy test. Đây là phát hiện quan trọng cần nêu trong phần giới hạn của báo cáo, thay vì chỉ nhìn CPU% để đánh giá “máy còn khỏe”.

7.3 Hardware & Resource Log

- Hardware report: theo dxdiag/systeminfo (thực hiện 1 lần, đính kèm riêng).
- Resource log: CSV log liên tục (5s/mẫu) chạy song song mỗi lần k6 chạy, cột CPU_Raw(%), CPU_Normalized(%), MemoryMB, kèm PID để đối chiếu — theo quy trình đã mô tả ở mục 7.1.

8. AI-Analysis Critique

(Nội dung mục này tổng hợp và mở rộng từ `review_and_fix.md`, đối chiếu với dữ liệu thực tế thu được.)

8.1 Load Test — VU đề xuất bởi AI có thực sự “an toàn thái quá”?

AI ban đầu đề xuất 50 VU cho Load Test — con số này **có cơ sở hợp lý**: mục tiêu của Load Test là quan sát hành vi ở tải “bình thường/kỳ vọng”, không phải tìm giới hạn. Tuy nhiên, kết quả thực tế (p95=21.92ms, p99=33.15ms — thấp hơn threshold đề ra tới **20–30 lần**) cho thấy 50 VU **chưa chạm đến bất kỳ áp lực nào** lên hệ thống. Nhận định trong `review_and_fix.md` rằng nên thử 100–200 VU là hợp lý, **nhưng cần lưu ý**: việc tăng VU cho Load Test làm mờ ranh giới giữa Load Test và Stress Test về mặt khái niệm — nếu tăng lên 100–200 VU, nên đồng thời **xác định lại threshold** dựa trên baseline mới (không giữ nguyên p95<500ms vốn được đặt cho 50 VU), và nêu rõ trong tài liệu rằng đây là “Load Test ở mức tải kỳ vọng cao hơn”, không phải kết hợp Load+Stress.

Khuyến nghị: giữ 50 VU làm baseline chính thức (đã có threshold xác nhận), bổ sung 1 lần chạy phụ ở 150 VU làm “extended load” để có thêm điểm dữ liệu, không thay thế bài Load Test gốc.

8.2 Spike Test — “Bug CPU=0%” và đề xuất tăng đỉnh spike

Về nghi ngờ CPU=0% là bug: đây **không phải bug** của script hay của Spike Test — như phân tích ở mục 7.2, đó là hệ quả tất yếu của (a) workload I/O-bound và (b) tần suất lấy mẫu 5s quá thừa so với biến động CPU thực tế trong vài giây. Không cần “fix” theo nghĩa sửa lỗi; cách cải thiện đúng là giảm interval lấy mẫu (xuống 1s) và/hoặc dùng rolling average, đã trình bày ở mục 7.1.

Về đề xuất tăng đỉnh spike lên 500-600 VU: có cơ sở thực nghiệm hợp lý — tỷ lệ lỗi ở đỉnh 400 VU là 0.00% (scenario main_workflow) và post_spike_duration cho thấy phục hồi gần như tức thời (p95=32ms), tức hệ thống còn “dư địa”. Tuy nhiên cần lưu ý: mục tiêu của Spike Test là quan sát **hành vi tại điểm gây đột ngột và khả năng phục hồi**, không phải tìm giới hạn tối đa (đó là việc của Stress Test). Đẩy đỉnh lên 600 VU có giá trị bổ sung, nhưng nên diễn giải kết quả dưới góc độ “khả năng chịu đựng cú sốc tải”, tách biệt với kết luận về giới hạn hệ thống.

8.3 Stress Test — Vấn đề cốt lõi: breaking point chưa được tìm thấy

Đây là phát hiện quan trọng nhất cần nêu bật trong báo cáo. Ở bậc tải cao nhất đã thử (800 VU, giữ 3 phút), hệ thống vẫn đạt: - checks_succeeded: 100.00% - http_req_failed{scenario:main_workflow}: 0.00% - Threshold rate<0.10: đạt với biên độ rất lớn (0.00% so với ngưỡng 10%)

Nhận định trong review_and_fix.md — “*vẫn chưa xác định được rằng máy mạnh hay do 800VUs vẫn còn ít*” — là **chính xác và là hạn chế thực sự của bài test này**, không phải điều “cho thấy tiềm năng vượt kỳ vọng AI” một cách đơn thuần. Về mặt phương pháp luận performance testing, một Stress Test **chỉ hoàn thành mục tiêu khi tìm được điểm hệ thống bắt đầu suy giảm/lỗi** (breaking point) — nếu không, kết luận đúng đắn duy nhất là “*chưa xác định được giới hạn trong phạm vi tải đã thử*,” không phải “*hệ thống mạnh hơn dự kiến*.” Hai khả năng cùng tồn tại và **báo cáo không thể phân biệt được** với dữ liệu hiện có:

1. Backend Node.js + SQLite thực sự chịu tải tốt ở quy mô single-machine demo (khả năng cao, vì kiến trúc đơn giản, không có network latency thật, chạy localhost).
2. Laptop test đủ mạnh để che khuất giới hạn thực của ứng dụng — 800 VU trên localhost tạo tải CPU/network không tương đương 800 người dùng thật qua internet.

Khuyến nghị bắt buộc trước khi kết luận báo cáo: chạy thêm ít nhất 1-2 bậc cao hơn (ví dụ 1200, 1600 VU) hoặc đến khi vus_max bị giới hạn bởi tài nguyên máy test (CPU/RAM của máy chạy k6, không phải server), và ghi nhận rõ **lý do dừng** (giới hạn ứng dụng, giới hạn máy test k6, hay giới hạn thời gian làm bài). Nếu không thể chạy thêm do giới hạn thời gian, báo cáo phải nêu rõ đây là **giới hạn đã biết (known limitation)** của kết quả Stress Test, không nên diễn giải thành kết luận tích cực về “tiềm năng của laptop.”

8.4 Endurance Test — chưa có phân tích trong review_and_fix.md, cần bổ sung

Nhận thấy review_and_fix.md chưa có nội dung cho Endurance Test. Dữ liệu thực tế cho thấy 1 phát hiện đáng chú ý mà bản đánh giá gốc còn thiếu: khi giữ nguyên 800 VU suốt 14 phút (khác với Stress Test chỉ giữ 800 VU trong ~3 phút cuối của một chuỗi tăng dần), latency các endpoint (đặc biệt checkout, login) **cao hơn rõ rệt** so với cùng mức VU trong Stress Test (xem so sánh ở mục 5.4). Đây là tín hiệu degradation theo thời gian dưới tải bền vững — **chính là mục tiêu cốt lõi mà Endurance Test cần phát hiện**, và là bằng chứng cho thấy việc chọn ENDURANCE_VU = 800 (bậc cao nhất của Stress Test, theo đúng hướng dẫn ở Checkpoint mục 11.2) là lựa chọn hợp lý — nó bộc lộ được hiệu ứng mà Stress Test (thời gian giữ tải ngắn hơn) không thấy rõ.

5 lỗi tại /api/cart chỉ xuất hiện ở bài Endurance (không xuất hiện ở Load/Stress/Spike) cũng củng cố giả thuyết liên quan đến userCarts lưu trong RAM — cần điều tra thêm liệu có phải do cấu trúc dữ liệu này phình to theo thời gian dưới tải bền vững kéo dài.

8.5 Monitoring script — quá trình debug là một phần giá trị của báo cáo

Diễn biến từ bug PID không cố định (Select-Object -First 1 chọn nhầm process) đến giải pháp đối chiếu PID thủ công qua Task Manager là một quá trình debug hợp lý và nên được trình bày như một phần của **phương pháp luận thu thập evidence**, không chỉ là ghi chú phụ — vì nó ảnh hưởng trực tiếp đến độ tin cậy của dữ liệu resource monitoring trong toàn bộ báo cáo (đã trình bày chi tiết ở mục 7.1).

9. Vấn đề công cụ & Threshold đề xuất (AI-Analysis)

9.1 k6 JSON raw vs JMeter .jtl

Đã giải quyết ở phần trao đổi trước khi làm báo cáo: k6 không tạo .jtl (định dạng độc quyền của JMeter), nhưng --out json tạo ra raw log ở mức sample/metric tương đương về **vai trò** (dùng cho hậu xử lý, truy vết), khác về **cấu trúc lưu trữ** (time-series theo từng metric, thay vì 1 dòng/request như .jtl). **Cần xác nhận với giảng viên** liệu k6 JSON + --summary-export có được chấp nhận thay thế .jtl hay không (mục việc còn cần làm, xem mục 11).

9.2 Threshold đề xuất cho các lần chạy tiếp theo

Dựa trên percentile thực tế thu được từ 4 lần chạy, đề xuất threshold theo 3 nhóm endpoint, tách biệt theo loại test:

Nhóm	Metric	Load (baseline, SLA)	Stress/Endurance (breaking-point test)	Spike (shock test)
Auth-heavy (login)	p95	< 50ms	fail rate < 10%	fail rate < 15%
Auth-heavy (login)	p99	< 100ms	—	—
Read-heavy (search, detail)	p95	< 30ms	fail rate < 10%	fail rate < 15%
Read-heavy (search, detail)	p99	< 60ms	—	—
Transactional (cart, checkout)	p95	< 100ms	fail rate < 10%	fail rate < 15%
Transactional (cart, checkout)	p99	< 200ms	—	—

Cơ sở đề xuất: ngưỡng Load lấy dư khoảng 2-3 lần so với p95/p99 thực đo ở 50 VU (mục 5.1), đủ biên độ cho biến động tự nhiên nhưng vẫn đủ chặt để phát hiện regression thực sự. Với Stress/Spike/Endurance, giữ nguyên triết lý threshold theo **tỷ lệ lỗi** (không theo latency tuyệt đối) vì mục tiêu các bài này là tìm giới hạn chịu tải chứ không phải đảm bảo SLA — cách tiếp cận này khớp với threshold đã cấu hình sẵn trong 4 script hiện tại (rate<0.10, rate<0.15, rate<0.05), nên giữ nguyên, chỉ bổ sung threshold Load ở trên cho các lần chạy Load Test mở rộng (100-150 VU, theo mục 8.1).

10. Giới hạn đã biết (Known Limitations)

1. **Breaking point thực sự của hệ thống chưa được xác định** (xem mục 8.3) — kết quả Stress Test hiện tại chỉ chứng minh hệ thống chịu được đến 800 VU trên localhost, không chứng minh đây là giới hạn tối đa.
2. **CPU monitoring chỉ đại diện, không toàn diện** — sampling 5s có thể bỏ sót các burst CPU ngắn (xem mục 7.2).
3. Định dạng raw log (JSON) khác .jtl yêu cầu của đề bài — cần xác nhận giảng viên (mục 9.1).
4. 5 lỗi tại /api/cart trong Endurance Test chưa được điều tra nguyên nhân gốc (chỉ mới có giả thuyết liên quan userCarts trong RAM).
5. SQL injection tại /api/products?search= đã biết nhưng **cố tình không khai thác** trong lúc test performance (dùng từ khóa “sạch”) — đây là giới hạn có chủ đích của phạm vi test, không phải thiếu sót.

11. Việc còn cần làm trước khi nộp bài

1. Xác nhận với giảng viên về định dạng output (.jtl vs k6 JSON/HTML) — mục 9.1.
2. Cân nhắc chạy thêm 1–2 bậc Stress Test cao hơn 800 VU để thu hẹp khoảng chưa xác định ở mục 8.3.
3. Điều tra 5 lỗi /api/cart trong Endurance Test.
4. Lập GitHub Issue cho bug lockout (đã thống nhất, xem mục 6) — hoàn tất phần ghi chú trong báo cáo, đính kèm link Issue khi có.
5. Hoàn thiện bảng ghi thời điểm reset DB giữa các lần chạy (reset-procedure.md) làm bằng chứng kèm phụ lục.

Phụ lục A — Tổng hợp threshold thực tế đã đạt

Test	Threshold cấu hình	Kết quả thực tế
Load	p(95)<500ms	21.92ms [PASS]
Load	p(99)<1000ms	33.15ms [PASS]
Load	http_req_failed rate<0.01	0.00% [PASS]
Load	login_fail_rate rate<0.01	0.00% [PASS]
Stress	http_req_failed{main_workflow} rate<0.10	0.00% [PASS]
Spike	http_req_failed{main_workflow} rate<0.15	0.00% [PASS]
Endurance	http_req_failed rate<0.05	0.00% [PASS]

Phụ lục B — Nguồn dữ liệu

- Console logs: load-test-console.txt, stress-test-console.txt, spike-test-console.txt, endurance-test-console.txt
- Summary exports: load-test-summary.json, stress-test-summary.json, spike-test-summary.json, endurance-test-summary.json
- Ghi chú review gốc: review_and_fix.md
- Bối cảnh kỹ thuật: CHECKPOINT.md